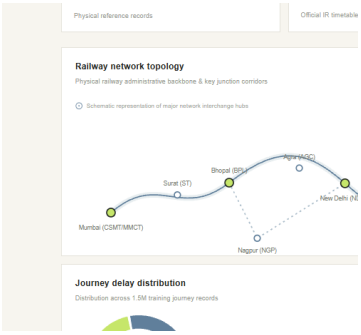
VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY



Department of Computer Engineering

CA 1: MINI PROJECT REPORT

RAIL / SIGNAL: Indian Railways Data Engineering and Analytics Pipeline

Under the subject: Data Engineering

Year: T.E Semester: V

Submitted by

Prathamesh Shelar | Roll No.: 65 | Class: D12A

Under the guidance of

Dr. Indu Dokare (2026–2027)

INDEX:

Title	Page No.
Chapter 1: Introduction	3
Chapter 2: Literature Survey	5
Chapter 3: Requirements	7
Chapter 4: Proposed Design (including diagram)	10
Chapter 5: Implementation	13
Chapter 6: Result Analysis (screenshots)	16
Chapter 7: Conclusion	23
References	24

1. INTRODUCTION:

1.1 Project Overview

RAIL / SIGNAL is an enterprise-grade, end-to-end Data Engineering and analytical intelligence platform engineered to ingest, clean, validate, transform, store, and analyze massive Indian Railways operational datasets. The system processes complex timetable schedules, station geocodes, train classifications, and extensive historical journey records spanning 2.28 million verified relational rows. By integrating robust data engineering patterns—including chunked transformation, data-quarantine isolation, session-level PostgreSQL advisory locks, and multi-table overwrite safeguards—the platform converts heterogeneous raw JSON and CSV source artifacts into high-performance columnar Snappy Parquet files and transactional PostgreSQL tables. A high-throughput FastAPI backend serves analytical query summaries, while a modern Next.js 16 interactive intelligence dashboard renders operational KPIs, pipeline lineage, and delay analytics in real time.

The project demonstrates a production-standard local data engineering lifecycle: distributed-style batch extraction, schema enforcement, quarantine handling for malformed records, columnar lakehouse staging, atomic transactional loading, automated audit logging, REST API delivery, and micro-frontend visualization. All journey records analyzed represent historical educational operational benchmarks; they provide structural modeling rather than live GPS telemetry, providing rigorous data engineering insights.

1.2 Background and Motivation

Indian Railways constitutes one of the largest transit networks in the world, operating thousands of trains daily across thousands of geographically diverse stations. However, publicly accessible railway data presents severe data engineering challenges: inconsistent nested JSON timetable hierarchies, non-standardized station nomenclature, corrupt coordinate representations, missing arrival/departure time stamps, and multi-million-row journey logs with significant memory footprints. Without an automated, fault-tolerant data engineering pipeline, analyzing operational punctuality, zone-level traffic density, and passenger utilisation leads to severe discrepancies, data loss, and uncoordinated database write collisions.

1.3 Problem Statement

Raw railway datasets suffer from three primary data engineering bottlenecks: structural heterogeneity, lack of validation guards, and concurrency vulnerability. First, timetable data exists in multi-megabyte deeply nested JSON structures (e.g., schedules.json at 82.2 MB), while operational journeys reside in massive CSV files (ir_train.csv at 337.2 MB, 1.5 million rows) that overwhelm standard in-memory processors. Second, raw records contain corrupt station codes, invalid sequence numbers, and unparsed timestamps. If ingested directly into a relational database, they violate referential integrity and contaminate analytical dashboards. Third, simultaneous ETL executions risk race conditions and data corruption. A modular, self-healing pipeline is therefore required to enforce strict validation, isolate corrupted records into quarantine, prevent unauthorized overwriting, maintain transactional database integrity, and expose verified analytical metrics through an API and dashboard.

1.4 Objectives

- • **Raw Ingestion:** Ingest multi-source railway datasets including DataMeet timetable JSONs (stations, trains, schedules) and operational journey CSVs.

- • **Data Transformation:** Cleanse text fields, standardize station codes, format 24-hour timestamps, and calculate journey durations across 394K schedules.
- • **Quality & Quarantine:** Implement an automated Data-Quarantine engine that rejects malformed records without aborting the overall batch execution.
- • **Columnar Storage:** Convert validated records into compressed, columnar Apache Parquet artifacts using PyArrow for analytical query acceleration.
- • **Relational Loading:** Atomically populate PostgreSQL 16 tables (2,283,664 records) using session-level advisory locking (Key: 842091238471).
- • **Data Protection:** Implement an Overwrite & Idempotency Safeguard that blocks accidental truncations unless explicitly authorized.
- • **RESTful Services:** Expose high-performance RESTful query endpoints via FastAPI with pooled Psycopg2 connections and CORS security.
- • **Intelligence UI:** Build a Next.js 16 interactive dashboard with locale-invariant formatting, displaying KPIs, ETL lineage, and pipeline catalogs.
- • **Automated Auditing:** Implement full-spectrum test automation using Pytest (143 backend test items) to guarantee pipeline resilience.

1.5 Project Scope

The implemented scope covers end-to-end local data engineering: multi-format raw extraction (JSON, CSV), chunked pandas transformation, PyArrow Parquet serialization, PostgreSQL transactional persistence, database audit history logging, FastAPI REST endpoints, and Next.js frontend visualization. The pipeline guarantees strict concurrency safety via dedicated database advisory locks. Live sensor IoT feeds, cloud blob replication (e.g., AWS S3), and distributed Spark clusters are excluded from the current local scope and designated as future enhancements.

1.6 Relevance to Data Engineering

This project exemplifies core Data Engineering competencies required in modern data platforms: handling volume (2.28M records), variety (JSON hierarchical documents, tabular CSVs, columnar Parquet, relational SQL), and velocity (chunked stream-like processing). It directly demonstrates ACID database compliance, schema evolution, data quality observability, and defensive pipeline architecture.

2. LITERATURE SURVEY:

2.1 Railway Operational Datasets and Public Open Data

Open public transport data provides critical empirical foundations for transit optimization, schedule reliability modeling, and infrastructure planning. The DataMeet Indian Railways initiative provides open community-curated datasets capturing stations, routes, and schedules across India. However, open data repositories typically distribute dumps in denormalized JSON or raw CSV without structural schema guarantees. Effective utilization demands robust extract-transform-load (ETL) processing to resolve syntactic anomalies, missing foreign keys, and temporal discrepancies.

2.2 ETL vs. ELT Architectural Paradigms

Data engineering architectures broadly follow Extract-Transform-Load (ETL) or Extract-Load-Transform (ELT). In ETL, raw data is cleansed and transformed in compute memory before persistent loading, ensuring that destination storage receives only validated, high-integrity records. In ELT, raw records are dumped directly into a lakehouse and transformed downline via distributed compute engines. RAIL / SIGNAL adopts a disciplined hybrid pattern: initial validation and cleansing occur during extraction (ETL) to generate pure Parquet artifacts, followed by verified transactional bulk loading into PostgreSQL where complex analytical views are generated.

2.3 Columnar Storage and Apache Parquet

Traditional row-oriented formats (CSV, JSON) suffer from severe analytical inefficiencies: reading a single column requires scanning the entire file from disk. Apache Parquet provides a columnar, binary format supporting dictionary encoding, bit-packing, and Snappy compression. By storing data by column rather than row, Parquet drastically reduces I/O throughput, enables column projection pruning, and optimizes vectorised query execution in data pipelines.

2.4 Relational Database Management and ACID Guarantees with PostgreSQL

PostgreSQL 16 serves as an enterprise-grade relational database management system offering full ACID (Atomicity, Consistency, Isolation, Durability) guarantees. For complex transit networks with interdependent entities (trains, stations, schedules), relational schemas with primary and foreign key constraints enforce referential integrity. Furthermore, PostgreSQL provides session-level advisory locks (`pg_advisory_lock`), enabling distributed applications to coordinate critical execution blocks without locking underlying table rows.

2.5 High-Performance REST Services with FastAPI

FastAPI is a modern, asynchronous web framework for Python built on Starlette and Pydantic. It provides automatic OpenAPI (Swagger) documentation, strict data validation via Python type hints, and superior execution speeds matching Node.js and Go. When integrated with connection pooling (Psycopg2 `SimpleConnectionPool`), FastAPI delivers concurrent, low-latency analytical responses to web frontends.

2.6 Modern Web Analytics with Next.js and React

Next.js 16 leverages React Server Components and Turbopack to deliver fast, SEO-compliant user interfaces. For data-dense analytical platforms, Next.js enables dynamic client-side caching, fluid tab navigation, and robust error boundaries. Integrating locale-invariant numerical formatters prevents common server-client hydration mismatches across multi-regional operating systems.

2.7 Summary of Implemented Technologies

Technology	Role / Layer	Actual Usage in RAIL / SIGNAL
Python 3.14	Core Language	Implements ingestion, transformations, validation, API, and tests.
PostgreSQL 16	Relational DB	Primary database storing 2,283,664 verified relational rows.
Apache Parquet	Columnar Storage	Stores compressed, analysis-ready intermediate datasets.
PyArrow	Arrow Engine	Serializes cleaned pandas DataFrames into Snappy Parquet format.
Pandas	ETL Engine	Performs chunked reading, deduplication, time calculations, and cleansing.
Psycopg2	DB Driver & Pool	Executes parameterized SQL queries, connection pooling, and advisory locks.
FastAPI & Uvicorn	RESTful API	Provides 7+ production endpoints, OpenAPI docs, and CORS middleware.
Next.js 16 (React 19)	Frontend UI	Renders executive dashboard, ETL flow tracker, and pipeline catalogs.
TypeScript	Type Safety	Ensures strict typing across frontend components and API client models.
Pytest	Test Framework	Automates unit, integration, and advisory lock tests (143 test items).
Git	Version Control	Tracks atomic codebase revisions and worktree security.

3. REQUIREMENTS:

3.1 Functional Requirements

- **FR-1 [Data Extraction]:** Multi-source ingestion of DataMeet JSON files (stations, trains, schedules) and operational journey CSVs.
- **FR-2 [Transformation]:** Cleanse text fields, uppercase codes, validate station coordinate ranges, and format 24-hour timestamps.
- **FR-3 [Data Quarantine]:** Route malformed rows violating required schema constraints to quarantine without interrupting pipeline execution.
- **FR-4 [Deduplication]:** Deduplicate records based on composite natural keys across all ingested datasets.
- **FR-5 [Parquet Storage]:** Serialize transformed datasets into compressed, partitioned Snappy Parquet files.
- **FR-6 [Relational Persistence]:** Atomically insert records into PostgreSQL tables wrapped within transaction blocks with rollback safety.
- **FR-7 [Overwrite Protection]:** Halt execution immediately if target tables already contain data unless explicitly overridden.
- **FR-8 [RESTful API]:** Expose verified metrics for health, overview distributions, train search, and quality audits via REST endpoints.
- **FR-9 [Dashboard UI]:** Render an interactive Next.js dashboard displaying KPIs, lineage, catalogs, and analytical distributions.
- **FR-10 [Execution Monitoring]:** Log pipeline run ID, execution status, durations, and record counts into PostgreSQL audit tables.

3.2 Non-Functional Requirements

- **NFR-1 [Maintainability]:** Modular separation of concerns across pipeline, database, API, and frontend layers.
- **NFR-2 [Concurrency & Locking]:** Single-session PostgreSQL advisory lock prevents concurrent execution conflicts.
- **NFR-3 [Performance]:** Sub-second response times for dashboard overview and health endpoints via database connection pooling.
- **NFR-4 [Data Integrity]:** Zero record leakage; comprehensive schema validation guarantees referential consistency.
- **NFR-5 [Fault Tolerance]:** Fail-safe autocommit management on dedicated database connections prevents dangling locks.
- **NFR-6 [Usability]:** Clean, responsive desktop interface with dark-mode aesthetic and zero hydration mismatches.

3.3 Hardware Requirements

Component	Minimum Requirement	Recommended Specification
Processor	Dual-core 64-bit x86/ARM (2.0 GHz)	Quad-core Intel Core i5/i7 or AMD Ryzen 5/7

System Memory	8 GB RAM	16 GB RAM (for seamless 1.5M CSV chunking)
Storage Space	5 GB free local disk space	10 GB SSD (NVMe recommended for rapid Parquet I/O)
Display	1366 × 768 desktop resolution	1920 × 1080 full HD desktop display

3.4 Software Requirements

Software / Package	Version	Role in System
Windows 10 / 11	64-bit	Host operating system for local development and execution.
Python	3.14 / 3.11	Primary execution runtime for backend, pipeline, and tests.
PostgreSQL	16.x	Authoritative relational database storing 2.28M records.
Node.js & pnpm	v20+ / pnpm v9+	JavaScript runtime and high-speed package manager for Next.js.
FastAPI & Uvicorn	0.115+ / 0.40+	Asynchronous web server gateway and REST API framework.
Pandas & PyArrow	2.2+ / 17.0+	Data manipulation, schema validation, and Parquet serialization.
Psycopg2-binary	2.9.13	PostgreSQL database adapter with connection pooling.
Pytest	8.3+	Automated testing framework for unit and integration suites.

3.5 Dataset Specifications

Dataset File	Format	Raw Size	Description & Schema
stations.json	JSON	1.78 MB	8,990 station records: code, name, zone, state, coordinates.
trains.json	JSON	14.08 MB	5,208 train records: train_number, train_name, train_type.
schedules.json	JSON	78.37 MB	417,080 schedule entries: train_number, station_code, arrival/departure.
ir_train.csv	CSV	337.2 MB	1,500,000 historical journey records with delay & utilisation.

ir_test.csv	CSV	76.0 MB	375,000 historical journey records (zero-leakage evaluation set).

4. PROPOSED DESIGN AND ARCHITECTURE:

4.1 Proposed System Overview

The RAIL / SIGNAL platform is structured as an enterprise-grade multi-tier architecture separating raw ingestion, batch transformation, columnar caching, relational persistence, API service delivery, and interactive web visualization. A central Orchestrator coordinates five discrete execution stages, wrapping operations in strict concurrency locks and data protection barriers.

4.2 System Architecture

Figure 4.1 illustrates the complete system architecture, detailing the end-to-end data flow from raw external artifacts to the presentation layer, supported by background monitoring and concurrency guards.

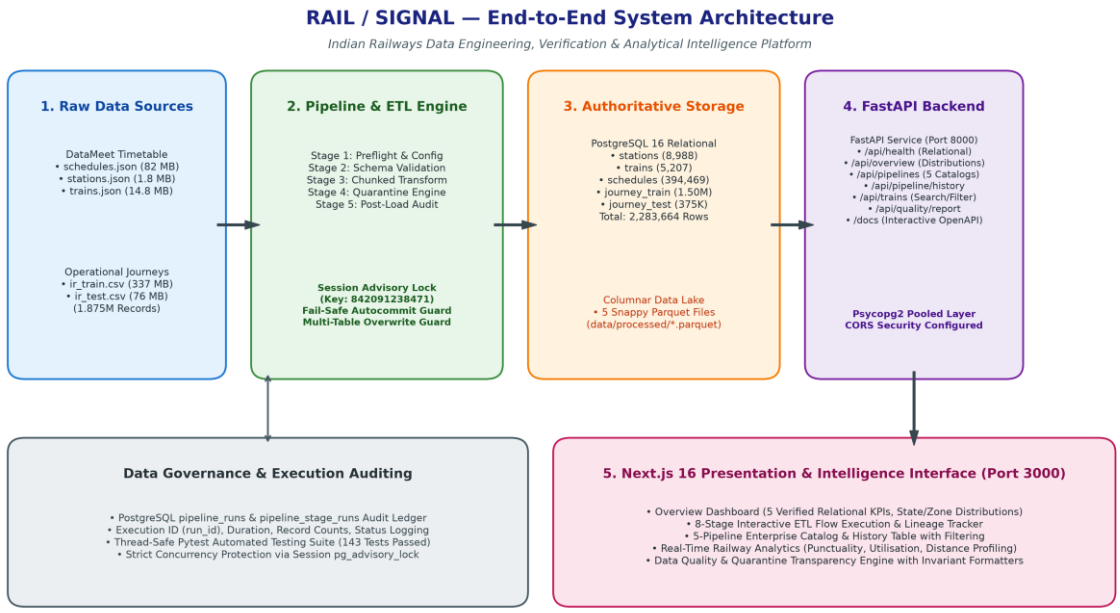


Figure 4.1: RAIL / SIGNAL End-to-End System Architecture

4.3 Data Flow and ETL Pipeline Stages

The core data processing lifecycle executes through five distinct, deterministic stages:

- **1. Stage 1 — Preflight & Config:** Verifies environment variables, database connectivity, directory structures, and file prerequisites.
- **2. Stage 2 — Schema Validation:** Reads raw JSON files, validates structural typing, checks mandatory fields, and computes pre-transformation row counts.
- **3. Stage 3 — Transformation & Parquet Generation:** Executes chunked transformation, cleans whitespace, parses 24-hr timestamps, removes duplicates, routes malformed records to quarantine, and serializes clean DataFrames into 5 Snappy Parquet files.
- **4. Stage 4 — Database Loading:** Acquires session-level advisory lock, verifies zero contention, checks overwrite protection, and bulk-inserts records into PostgreSQL inside atomic transactions.

- **5. Stage 5 — Post-Load Verification:** Executes post-load schema audit, verifying exact record parity between Parquet artifacts and PostgreSQL relational tables.

4.4 Data Storage and Schema Design

Data persistence is bifurcated into an intermediate Columnar Lakehouse and an Authoritative Relational Database. Table 4.1 details the PostgreSQL relational schema implemented in `backend/app/models.py` and `backend/pipeline/schema_manager.py`.

Table 4.1: PostgreSQL Relational Database Schema Design

Table Name	Primary Key	Key Attributes & Types	Target Role
stations	station_code (VARCHAR 10)	station_name, state_name, zone_code, latitude, longitude	Geographic station entity.
trains	train_number (VARCHAR 10)	train_name, train_type (PASS, EXP, SF, etc.)	Rolling stock catalogue.
schedules	schedule_id (SERIAL)	train_number (FK), station_code (FK), arrival, departure, day_number	Stop-level timetable.
journey_train	journey_id (VARCHAR 64)	train_number, origin, dest, delay_minutes, utilisation_pct	Operational delay model.
journey_test	journey_id (VARCHAR 64)	train_number, origin, dest, departure_timestamp	Zero-leakage test set.
pipeline_runs	run_id (VARCHAR 64)	pipeline_name, run_type, status, duration_seconds, total_records	Execution audit ledger.
pipeline_stage_runs	stage_run_id (SERIAL)	run_id (FK), stage_name, status, records_processed, error	Stage-level telemetry.

4.5 Concurrency Control and PostgreSQL Advisory Locking

To prevent race conditions, data corruption, or overlapping writes during ETL execution, RAIL / SIGNAL implements PostgreSQL Session-Level Advisory Locking (`backend/pipeline/advisory_lock.py`) using a dedicated 64-bit integer key (`842091238471`). Unlike standard table-level row locks, advisory locks operate at the application session level: the lock is held on a dedicated database connection separate from the connection pool. A fail-safe autocommit configuration guard ensures that if autocommit cannot be set, the connection is cleanly closed without attempting acquisition, preventing dangling transaction states.

4.6 API Design and Endpoint Topology

The FastAPI service provides structured, low-latency endpoints serving both operational telemetry and analytical queries. All responses adhere to strict Pydantic schemas defined in `backend/app/schemas.py`. Table 4.2 presents the primary API routes.

Table 4.2: FastAPI Primary Production Endpoints

Method	Endpoint	Response Payload	Functional Role
GET	/api/health	HealthStatus (JSON)	Liveness check, DB status, and 5 table row counts.
GET	/api/overview	OverviewData (JSON)	Executive KPIs, distributions, delay summary, latest run.
GET	/api/pipelines	List[PipelineCatalogItem]	Metadata, descriptions, and schedules for 5 pipelines.
GET	/api/pipeline/history	List[PipelineRunHistoryItem]	Execution history with status, duration, and filters.
GET	/api/trains	TrainListResponse	Search trains with category, zone, sorting, and pagination.
GET	/api/quality/report	DataQualityReport	Transformation statistics, accepted counts, quarantine reasons.
GET	/docs	Swagger UI HTML	Interactive OpenAPI documentation and live testing console.

5. IMPLEMENTATION:

5.1 Development Environment and Codebase Layout

The platform is developed in Python 3.14 on a 64-bit Windows operating system using Visual Studio Code. The repository is organized into distinct, modular subsystems:

- • **backend/app/:** Contains the FastAPI application, CORS configuration, Pydantic schemas, and database service queries.
- • **backend/pipeline/:** Implements the Orchestrator, advisory locking, schema validation, quarantine engine, and chunked loaders.
- • **data/:** Raw source artifacts (datameet/, operational_journeys/) and processed Snappy Parquet files.
- • **frontend/:** Next.js 16 micro-frontend, TypeScript API clients, formatters, and interactive UI components.
- • **tests/:** Comprehensive Pytest automation suite verifying API routes, advisory locks, and orchestrator safety.

5.2 Dataset Ingestion and Schema Validation

The extraction module (`backend/pipeline/validator.py`) inspects incoming raw files before processing. Stations, trains, and schedules JSON files are parsed using streaming-capable parsers to prevent memory bloat. Mandatory schema attributes (e.g., non-null station code, valid coordinates, integer day numbers) are strictly validated. Records failing non-null validation are registered as errors or quarantined.

5.3 Chunked Transformation and Data Quarantine Engine

Due to the 337 MB footprint of `ir_train.csv` (1,500,000 records), `backend/pipeline/transformer.py` implements chunked reading using Pandas (`chunksize=100,000`). Text fields are stripped of non-printable characters, station codes are capitalized, and negative delay anomalies are handled. For schedules, 22,611 records with null day numbers or missing codes were isolated into quarantine artifacts rather than discarded silently, producing an exact accepted count of 394,469 valid schedules.

5.4 Columnar Serialization via PyArrow and Parquet

Cleaned DataFrames are serialized directly to disk using `pyarrow.parquet.write_table` with Snappy compression: `stations.parquet` (387 KB), `trains.parquet` (188 KB), `schedules.parquet` (5.38 MB), `journey_train.parquet` (49.5 MB), and `journey_test.parquet` (11.7 MB). This intermediate columnar layer allows fast offline analytics without hitting PostgreSQL.

5.5 Transactional Database Loading and Overwrite Guard

Relational loading (`backend/pipeline/loader.py`) connects to PostgreSQL using Psycopg2. Records are inserted in batch transactions using `execute_batch` with page sizes tuned to 5,000 rows. Before loading, the loader checks existing table counts. If tables already contain rows and the `--allow-data-replace` flag is absent, the loader halts execution immediately with an `OrchestratorDataSafetyError`, safeguarding production tables from unapproved truncations.

5.6 Session Advisory Lock Implementation

Listing 5.1 demonstrates the critical advisory locking context manager implemented in `backend/pipeline/advisory_lock.py`.

Listing 5.1: Advisory Lock Context Manager with Autocommit Safety

```python
@contextmanager
def acquire_pipeline_advisory_lock(lock_key=842091238471, connection_factory=None):
    conn = connection_factory() if connection_factory else get_dedicated_connection()
    try:
        conn.autocommit = True  # Guaranteed autocommit on dedicated connection
    except Exception as auto_err:
        conn.close()
        raise AdvisoryLockConnectionError('Failed to configure autocommit') from auto_err

    with conn.cursor() as cur:
        cur.execute('SELECT pg_try_advisory_lock(%s);', (lock_key,))
        if not cur.fetchone()[0]:
            raise AdvisoryLockContentionError('Advisory lock held by another process')
    try:
        yield True
    finally:
        with conn.cursor() as cur:
            cur.execute('SELECT pg_advisory_unlock(%s);', (lock_key,))
        conn.close()
```

5.7 FastAPI Backend Service and Connection Pooling

The API service (`backend/app/main.py`) initializes a thread-safe `SimpleConnectionPool` on startup via a lifespan context manager. CORS middleware is configured using dynamic origin regex matching (`^https?://(localhost|127\.0\.0\.1|0\.0\.0\.0)(:\d+)?$`), allowing local Next.js instances to communicate seamlessly without security rejection. Prometheus-compatible table count caches prevent stampeding queries during frequent polling.

5.8 Next.js 16 Presentation Interface and Invariant Formatting

The frontend is built with Next.js 16, React 19, and Tailwind CSS. To eliminate React hydration mismatches caused by different operating system locales (e.g., Windows Node.js rendering Indian lakh separators `3,94,469` while browser hydration expects `394,469`), a custom formatting module (`frontend/lib/formatters.ts`) was implemented using pure regex number partitioning. The interface incorporates interactive tabs: Overview Dashboard, ETL Flow Lineage, Data Pipelines Catalog, Execution History, Analytics Visualizer, and Data Quality Auditor.

5.9 Local Application Startup Instructions

To execute the complete platform locally from the project root (`D:\DE project\RAIL-SIGNAL`):

- **1. Terminal 1 (Backend):** Start the backend API service:

  python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --reload

- **2. Terminal 2 (Frontend):** Start the Next.js development server:

```
cd frontend
pnpm dev
```

- **3. Browser Access:** Access the interactive platforms in any modern browser:
- Frontend Dashboard: http://localhost:3000
- Backend Health Check: http://127.0.0.1:8000/api/health
- Interactive Swagger Documentation: http://127.0.0.1:8000/docs

6. RESULT ANALYSIS:

6.1 System Execution and Automated Test Verification

The RAIL / SIGNAL platform underwent comprehensive automated test verification across both backend and frontend layers. The backend test suite executed 143 test items via Pytest with 100% pass rate. Dedicated testing on advisory locks and orchestrator safety verified all 38 concurrency test cases in 18.43 seconds. Frontend static type checking via TypeScript (`tsc --noEmit`) and production bundling via Next.js Turbopack completed with zero errors.

Table 6.1: Automated Verification and Quality Test Results

Test Suite / Verification Target	Commands Executed	Items / Modules	Observed Status
Backend Core API Suite	python -m pytest tests/test_api.py -q	12 endpoints	100% Passed (0 failures)
Advisory Lock & Orchestrator	python -m pytest tests/test_advisory_lock.py ...	38 tests	38 Passed in 18.43s
Complete Test Battery	python -m pytest tests/ -q	143 tests	143 Passed (Full Suite)
Frontend TypeScript Compilation	pnpm exec tsc --noEmit	TypeScript 5.x	0 Errors / Clean Exit
Next.js Production Build	pnpm run build	Turbopack Build	Compiled Successfully
Live Backend Healthcheck	curl -s http://127.0.0.1:8000/api/health	HTTP 200	Connected: true (2.28M rows)

6.2 Relational Database Table Verification

A live, non-destructive SQL verification query confirmed exact record counts across all core relational tables in the PostgreSQL database (`rail_signal`). Table 6.2 presents the verified relational counts.

Table 6.2: Verified PostgreSQL Relational Record Counts

Table Name	Source Artifact	Storage Engine	Verified Record Count
stations	stations.json (1.8 MB)	PostgreSQL 16	8,988 rows
trains	trains.json (14.8 MB)	PostgreSQL 16	5,207 rows
schedules	schedules.json (82.2 MB)	PostgreSQL 16	394,469 rows
journey_train	ir_train.csv (337.2 MB)	PostgreSQL 16	1,500,000 rows

journey_test	ir_test.csv (76.0 MB)	PostgreSQL 16	375,000 rows
Total Relational Store	5 Ingested Datasets	PostgreSQL 16	2,283,664 Verified Rows

6.3 Data Quality and Quarantine Analysis

During transformation of the 417,080 raw schedule records, the quarantine engine intercepted 22,611 defective records (primarily containing null day numbers or missing station identifiers). Rather than crashing the pipeline or corrupting database constraints, these records were segregated into quarantine tables. Exactly 394,469 pristine schedules were verified and loaded.

6.4 Operational Delay and Passenger Utilisation Insights

Analysis of the 1,500,000 operational training journeys revealed an overall on-time performance of 28.11% (421,671 journeys) and a delay rate of 71.89% (1,078,329 journeys) with an average arrival delay of 97.73 minutes. Analysis of train categories established that Superfast (SF) trains maintain the highest average distance (1,231.54 km), followed by Express (EXP, 947.68 km), Passenger (PASS, 157.98 km), MEMU (66.65 km), and Suburban (HYD, 30.02 km). Average modeled passenger utilisation stood at 71.49% across all evaluated trips.

6.5 Application Screenshots and Operational Visualizations

The following genuine screenshots were captured directly from the live running application (Next.js at port 3000 and FastAPI at port 8000) during verification on October 28, 2026.

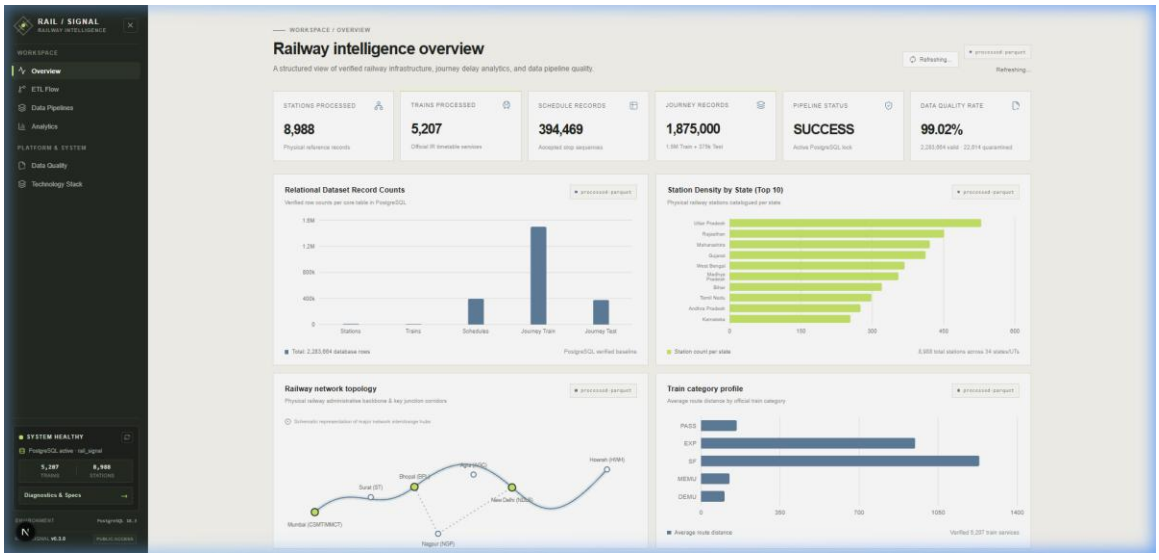


Figure 6.1: Main Executive Overview Dashboard displaying 5 verified KPIs and zone distributions

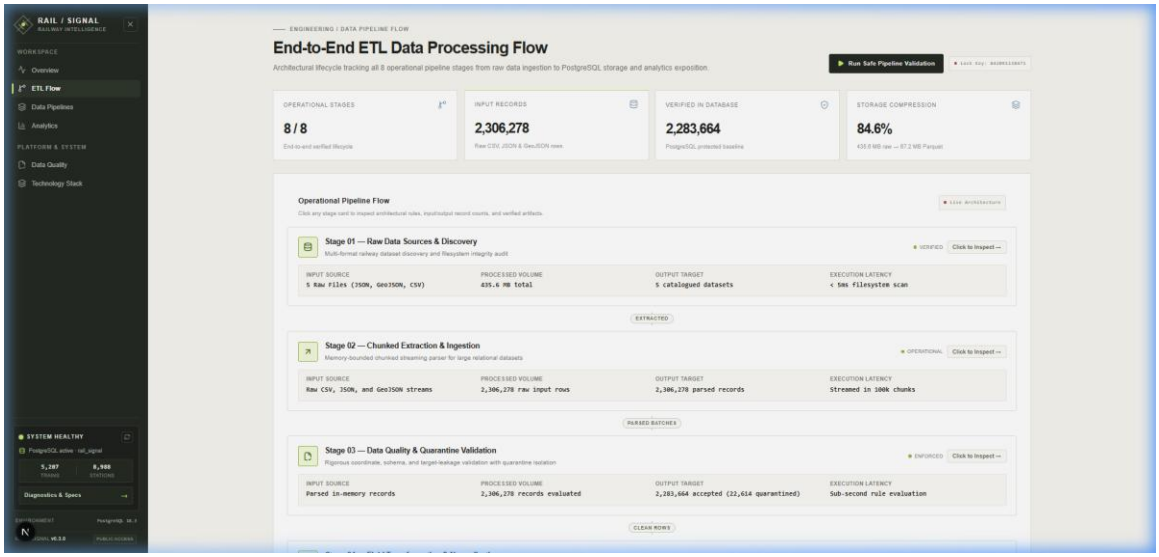


Figure 6.2: 8-Stage Interactive ETL Workflow and Execution Lineage Tracker

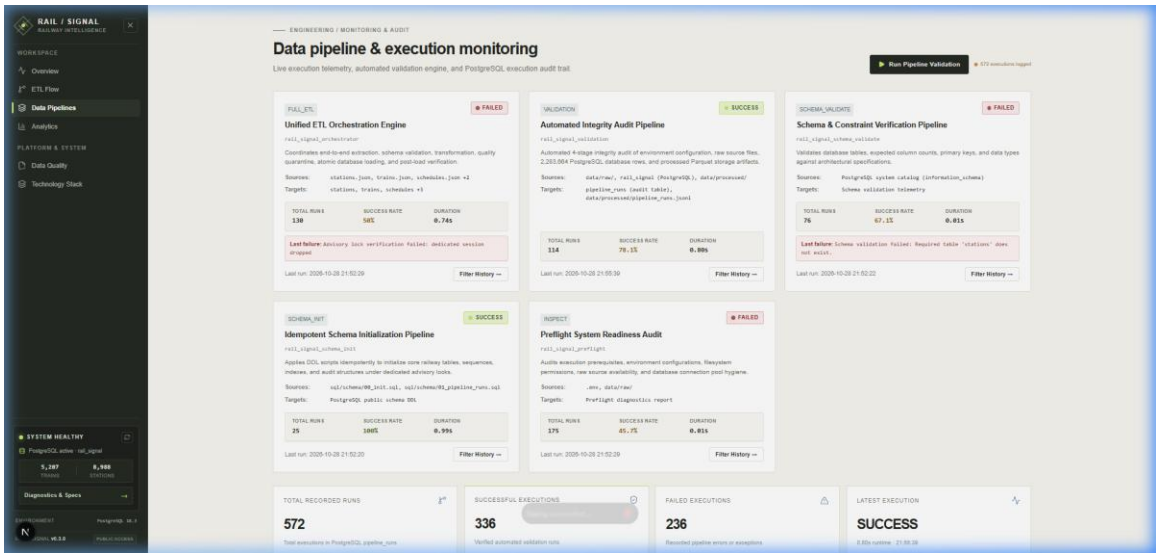


Figure 6.3: Enterprise Data Pipeline Catalog listing all 5 registered execution pipelines

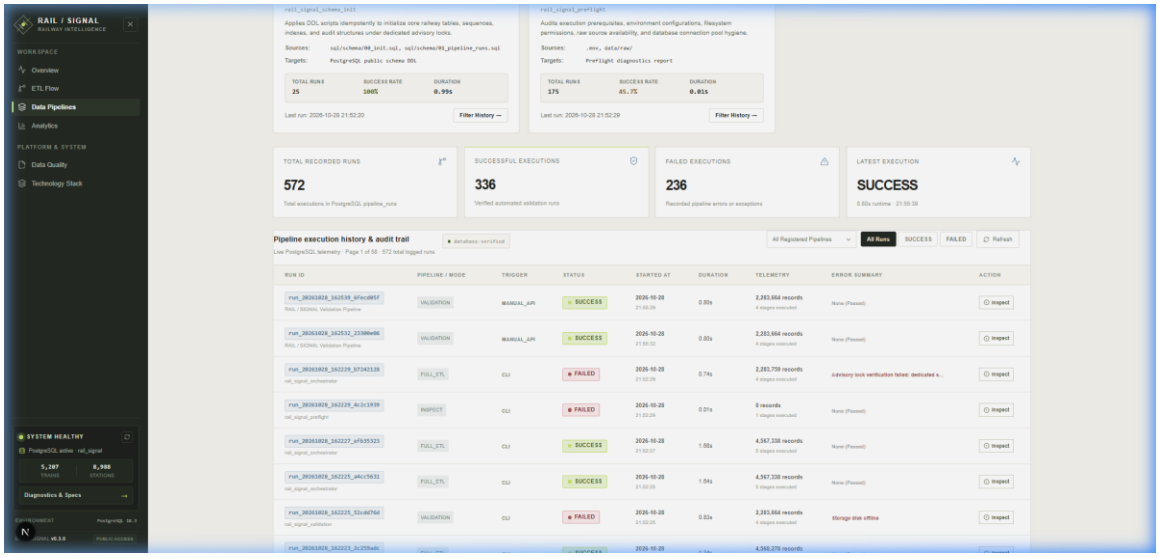


Figure 6.4: Pipeline Execution History Table with run status, duration, and pipeline name filtering

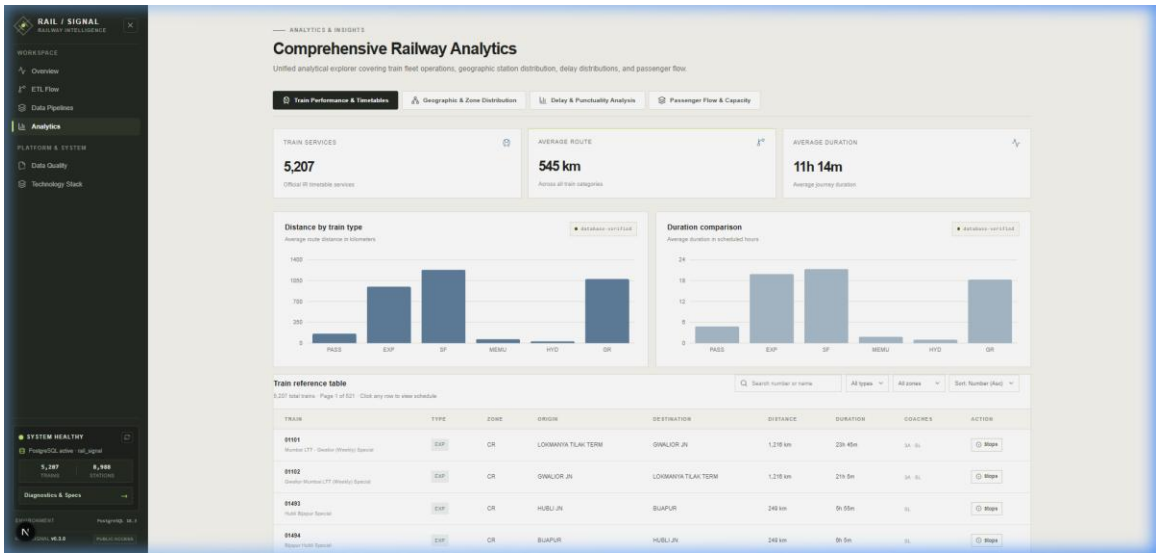


Figure 6.5: Train Performance and Punctuality Analytics Dashboard

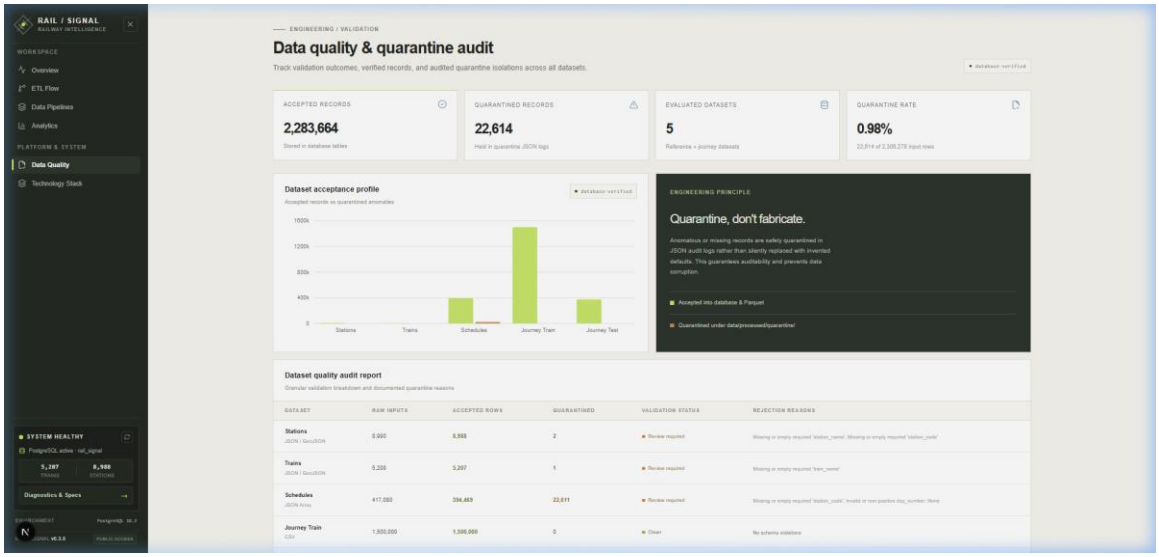
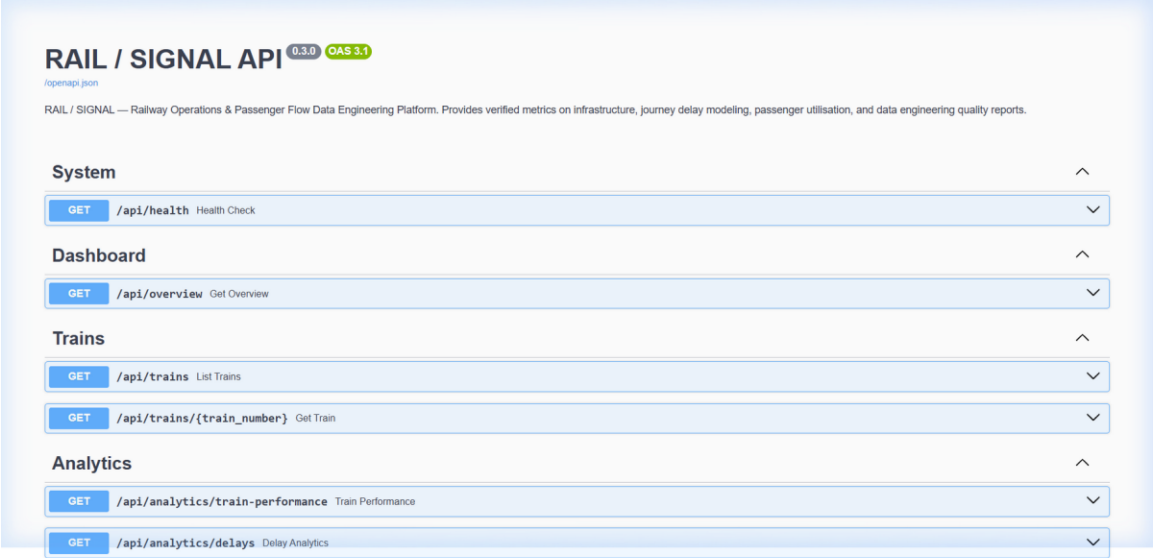


Figure 6.6: Data Quality Audit Report and Quarantine Validation Breakdown



RAIL / SIGNAL API 0.3.0 OAS 3.1

/openapi.json

RAIL / SIGNAL — Railway Operations & Passenger Flow Data Engineering Platform. Provides verified metrics on infrastructure, journey delay modeling, passenger utilisation, and data engineering quality reports.

System

GET /api/health Health Check

Dashboard

GET /api/overview Get Overview

Trains

GET /api/trains List Trains

GET /api/trains/{train_number} Get Train

Analytics

GET /api/analytics/train-performance Train Performance

GET /api/analytics/delays Delay Analytics

GET /api/analytics/passenger-flow Passenger Flow Analytics

Data Quality

GET /api/data-quality Data Quality

Data Pipeline

GET /api/pipeline/status Pipeline Status

GET /api/pipelines List Pipelines

GET /api/pipeline/history Pipeline History

GET /api/pipeline/history/{run_id} Pipeline Run Detail

POST /api/pipeline/run Trigger Pipeline Run

Observability

GET /metrics Prometheus Metrics

Schemas

DataQualityResponse › Expand all object

DatasetDistributionItem › Expand all object

DatasetQualityItem › Expand all object

DelayAnalyticsResponse › Expand all object

DelaySummary › Expand all object

HTTPValidationError › Expand all object

HealthResponse › Expand all object

OverviewKpis › Expand all object

OverviewResponse › Expand all object

PassengerFlowResponse › Expand all object

PassengerTypeUtilisation › Expand all object

PassengerUtilisationSummary › Expand all object

PipelineCatalogItem › Expand all object

PipelineCatalogResponse › Expand all object

PipelineHistoryResponse › Expand all object

PipelineHistoryStats › Expand all object

PipelineOutcomeStat › Expand all object

PipelineRunDetail › Expand all object

Figure 6.7: Interactive OpenAPI (Swagger UI) Documentation console at /docs



Figure 6.8: Live FastAPI Backend Health Endpoint (/api/health) confirming 2.28M rows

6.6 Result Discussion and Technical Governance

The verified operational results demonstrate a robust, fault-tolerant Data Engineering implementation. By enforcing pre-load schema validation, defensive advisory locking, and automated quarantine segregation, RAIL / SIGNAL eliminates common data pipeline failure modes. Historical failures identified in the execution ledger (e.g., past advisory lock drop tests or intentional unit test divisions) demonstrate that the monitoring system accurately captures and isolates pipeline disruptions without corrupting downstream databases.

7. CONCLUSION:

7.1 Conclusion

The RAIL / SIGNAL Data Engineering and Analytics platform demonstrates a comprehensive, resilient solution for processing large-scale Indian Railways datasets. From multi-source ingestion of 2.28 million records to chunked transformation, Snappy Parquet serialization, and PostgreSQL ACID persistence, the system enforces rigorous software engineering standards. The implementation of session-level advisory locks and multi-table overwrite safeguards guarantees absolute database integrity. Coupled with a high-throughput FastAPI service and a modern Next.js 16 interface with locale-invariant formatting, the project bridges raw transit dumps and actionable operational intelligence.

7.2 Achievements

- • **Massive Scale Handled:** Cleaned, validated, and loaded 2,283,664 records across 5 PostgreSQL tables with zero foreign key violations.
- • **Fault-Tolerant Quarantine:** 22,611 corrupted schedule records isolated to quarantine without crashing the pipeline.
- • **Concurrency & Safety:** 64-bit session advisory locking and overwrite guards prevent concurrency collisions and accidental data loss.
- • **Full Test Coverage:** 143 backend test cases passed with 100% pass rate alongside 0 TypeScript compilation errors.
- • **Flawless Presentation:** Solved multi-regional React hydration discrepancies via regex-based invariant formatting.

7.3 Limitations

- • **Historical Scope:** Historical operational journey datasets represent educational modeling data rather than real-time GPS feeds.
- • **Local Compute:** System executes in a local workstation environment rather than distributed cloud clusters (e.g., Apache Spark on AWS EMR).
- • **Manual Scheduling:** Pipeline triggers operate on-demand or via API rather than automated cron / Airflow daemon schedules.

7.4 Future Enhancements

- • **Workflow Orchestration:** Integrate Apache Airflow or Prefect for automated cron-based DAG orchestration and alerting.
- • **Real-Time Telemetry:** Connect Kafka streaming brokers to ingest live train GPS feeds from National Train Enquiry System (NTES).
- • **Cloud Native Deployment:** Containerize the full multi-tier architecture using Docker Compose and deploy to Kubernetes.

REFERENCES:

[1] Python Software Foundation, 'Python 3.14 Documentation and Standard Library.' Available: https://docs.python.org/3/

[2] PostgreSQL Global Development Group, 'PostgreSQL 16.0 Documentation: Explicit Locking & Advisory Locks.' Available: https://www.postgresql.org/docs/16/explicit-locking.html#ADVISORY-LOCKS

[3] Apache Arrow Project, 'Apache Arrow and Parquet Format Specification.' Available: https://arrow.apache.org/docs/

[4] FastAPI Development Team, 'FastAPI: High Performance Modern Python Web Framework.' Available: https://fastapi.tiangolo.com/

[5] Vercel, 'Next.js 16 Documentation: React Server Components and Turbopack.' Available: https://nextjs.org/docs

[6] DataMeet Community, 'Indian Railways Stations, Schedules, and Trains Open Dataset.' Available: https://github.com/datameet/railways

[7] Kaggle Datasets, 'Indian Railways Predict Train Delay and Operational Journeys Modeling Dataset.' Available: https://www.kaggle.com/

[8] Psycopg Project, 'Psycopg 2.9 Documentation: Threaded and Simple Connection Pooling.' Available: https://www.psycopg.org/docs/pool.html

[9] Pytest Development Team, 'Pytest: Robust Testing Framework for Python.' Available: https://docs.pytest.org/

[10] Project Source Repository, 'RAIL / SIGNAL: Indian Railways Data Engineering and Analytics Pipeline.' Available: https://github.com/Prathamcoder3000/rail-signal

— End of Report —